```
%%writefile kkr.c
#include <stdio.h>
#include <string.h>

int main() {
char rule[100];

printf("Enter production rule (e.g., A->Aa|b): ");
if (scanf("%99s", rule) != 1) return 1;

char lhs = rule[0]; // Left-hand side non-terminal character


if (rule[1] == '-' && rule[2] == '>' && rule[3] == lhs) {
char *pipe = strchr(rule, '|');

if (pipe != NULL) {
printf("\nOriginal Rule: %s\n", rule);
printf("\nRules After Eliminating Left Recursion:\n");
printf("%c -> %s%c'\n", lhs, pipe + 1, lhs);
printf("%c' -> ", lhs);
for (char *ptr = &rule[4]; ptr < pipe; ptr++) {
putchar(*ptr); // Prints alpha string characters one by one
}
printf("%c' | e\n", lhs);
} else {
printf("\nFormat Error: Missing option separator pipeline ('|').\n");
}
} else {
printf("\nNo direct left recursion detected in this rule.\n");
}
return 0;
}
```

```
Writing kkr.c
```

```
%%shell
gcc kkr.c
./a.out
```

```
Enter production rule (e.g., A->Aa|b): A->Aa|b

Original Rule: A->Aa|b

Rules After Eliminating Left Recursion:
A -> bA'
A' -> aA' | e
```

```
%%writefile follow.c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

// your FOLLOW set code here


int count;
char prodn[10][10];
char follow[10];
int n;

void FOLLOW(char c);

int main() {
    int i, choice;
    char c;

    printf("How many productions ? : ");
    scanf("%d", &count);

    printf("Enter %d productions epsilon = $ :\n\n", count);
    for (i = 0; i < count; i++)
        scanf("%s", prodn[i]);

    do {
        n = 0;
        printf("\nElement : ");
        scanf(" %c", &c);   // space before %c skips newline

        FOLLOW(c);

        printf("\nFOLLOW(%c) = { ", c);
        for (i = 0; i < n; i++)
            printf("%c ", follow[i]);
        printf("}\n");

        printf("Press 1 to continue : ");
        scanf("%d", &choice);

    } while (choice == 1);

    return 0;
}

void addToFollow(char c) {
    int i;
    for (i = 0; i < n; i++) {
        if (follow[i] == c)
            return;
```

```
    }
    follow[n++] = c;
}

void FOLLOW(char c) {
    int i, j;
    char next;

    /* Start symbol gets # */
    if (prodn[0][0] == c)
        addToFollow('#');

    for (i = 0; i < count; i++) {
        for (j = 2; prodn[i][j] != '\0'; j++) {
            if (prodn[i][j] == c) {
                /* If c is not the last symbol */
                if (prodn[i][j + 1] != '\0') {
                    next = prodn[i][j + 1];

                    /* If next symbol is a terminal */
                    if (islower(next))
                        addToFollow(next);

                    /* If next symbol is epsilon */
                    else if (next == '$') {
                        if (prodn[i][0] != c)
                            FOLLOW(prodn[i][0]);
                    }

                    /* If next symbol is a non-terminal */
                    else {
                        /* Find FIRST(next) */
                        for (int k = 0; k < count; k++) {
                            if (prodn[k][0] == next) {
                                if (prodn[k][2] != '$')
                                    addToFollow(prodn[k][2]);
                            }
                        }
                    }
                } else {
                    /* c is the last symbol, so add FOLLOW of LHS */
                    if (prodn[i][0] != c)
                        FOLLOW(prodn[i][0]);
                }
            }
        }
    }
}
```

Writing follow.c

```
%%shell
gcc follow.c -o follow
./follow

How many productions ? : 3
Enter 3 productions epsilon = $ :

 $ :
E->E+T|T T->T*F|F F->(E)|id

Element :
FOLLOW(T) = { }
Press 1 to continue :



%%writefile kkr.c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 20

char prod[MAX][20];
char first[26][20], follow[26][20];
int n;

void add(char *s, char c)
{
    if (!strchr(s, c)) {
        int l = strlen(s);
        s[l] = c;
        s[l + 1] = '\0';
    }
}

int isNT(char c)
{
    return (c >= 'A' && c <= 'Z');
}

void FIRST(char nt)
{
    int i, j;
    for (i = 0; i < n; i++) {
        if (prod[i][0] != nt)
            continue;

        char *rhs = prod[i] + 2;

        if (rhs[0] == '#') {
```

```
            add(first[nt - 'A'], '#');
        }
        else if (!isNT(rhs[0])) {
            add(first[nt - 'A'], rhs[0]);
        }
        else {
            char x = rhs[0];

            if (first[x - 'A'][0] == '\0')
                FIRST(x);

            for (j = 0; first[x - 'A'][j]; j++)
                add(first[nt - 'A'], first[x - 'A'][j]);
        }
    }
}

void FOLLOW()
{
    int i, j, k;

    add(follow[prod[0][0] - 'A'], '$');

    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            char A = prod[i][0];
            char *rhs = prod[i] + 2;

            for (j = 0; rhs[j]; j++) {
                if (isNT(rhs[j])) {
                    char B = rhs[j];

                    if (rhs[j + 1] != '\0') {
                        char x = rhs[j + 1];

                        if (!isNT(x))
                            add(follow[B - 'A'], x);
                        else {
                            for (int m = 0; first[x - 'A'][m]; m++)
                                if (first[x - 'A'][m] != '#')
                                    add(follow[B - 'A'],
                                        first[x - 'A'][m]);
                        }
                    }
                    else {
                        for (int m = 0; follow[A - 'A'][m]; m++)
                            add(follow[B - 'A'],
                                follow[A - 'A'][m]);
                    }
                }
```

```
            }
        }
    }
}

int main()
{
    int i, j;

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (use # for epsilon):\n");

    for (i = 0; i < n; i++)
        scanf("%s", prod[i]);

    /* FIRST */
    for (i = 0; i < n; i++)
        FIRST(prod[i][0]);

    /* FOLLOW */
    FOLLOW();

    printf("\nFIRST:\n");
    for (i = 0; i < n; i++) {
        char A = prod[i][0];

        if (i == 0 || prod[i - 1][0] != A)
            printf("FIRST(%c) = {%s}\n",
                   A, first[A - 'A']);
    }

    printf("\nFOLLOW:\n");
    for (i = 0; i < n; i++) {
        char A = prod[i][0];

        if (i == 0 || prod[i - 1][0] != A)
            printf("FOLLOW(%c) = {%s}\n",
                   A, follow[A - 'A']);
    }

    /* LL(1) Parsing Table */
    printf("\nLL(1) PARSING TABLE\n");
    printf("--------------------\n");

    for (i = 0; i < n; i++) {
        char A = prod[i][0];

        if (i > 0 && prod[i - 1][0] == A)
```

```
            continue;

        printf("\n%c : ", A);

        for (j = 0; j < n; j++) {
            if (prod[j][0] != A)
                continue;

            char *rhs = prod[j] + 2;

            if (rhs[0] == '#') {
                for (int k = 0; follow[A - 'A'][k]; k++)
                    printf("[%c,# -> #] ",
                           follow[A - 'A'][k]);
            }
            else if (!isNT(rhs[0])) {
                printf("[%c,%c -> %s] ",
                       A, rhs[0], rhs);
            }
            else {
                char B = rhs[0];

                for (int k = 0; first[B - 'A'][k]; k++)
                    if (first[B - 'A'][k] != '#')
                        printf("[%c,%c -> %s] ",
                               A, first[B - 'A'][k], rhs);
            }
        }
    }

    printf("\n");
    return 0;
}
```

```
Overwriting kkr.c
```

```
%%shell
gcc kkr.c
./a.out
```

```
Enter number of productions: 7
Enter productions (use # for epsilon):
E=TR R=+TR R=# T=FY Y=*FY Y=# F=(E) F=i

FIRST:
FIRST(E) = {(}
FIRST(R) = {+#}
FIRST(T) = {(}
FIRST(Y) = {*#}
FIRST(F) = {(}
```

```
FOLLOW:
FOLLOW(E) = {$)}
FOLLOW(R) = {$)}
FOLLOW(T) = {+}
FOLLOW(Y) = {+}
FOLLOW(F) = {*}

LL(1) PARSING TABLE
-------------------

E : [E,( -> TR]
R : [R,+ -> +TR] [$,# -> #] [),# -> #]
T : [T,( -> FY]
Y : [Y,* -> *FY] [+,# -> #]
F : [F,( -> (E)]



%%writefile shift_reduce.c
#include <stdio.h>
#include <string.h>

int main()
{
    char stack[50], input[50];
    int top = -1, i = 0;

    printf("Enter input string: ");
    scanf("%s", input);

    printf("\nSTACK\tINPUT\tACTION\n");

    while (input[i] != '\0') {
        stack[++top] = input[i++];
        stack[top + 1] = '\0';

        printf("%s\t%s\tSHIFT\n", stack, input + i);

        if (top >= 1 &&
            stack[top - 1] == 'i' &&
            stack[top] == 'd') {
            top -= 1;
            stack[top] = 'E';
            stack[top + 1] = '\0';

            printf("%s\t%s\tREDUCE E->id\n",
                   stack, input + i);
        }

        if (top >= 2 &&
            stack[top - 2] == 'E' &&
```

```
            stack[top - 1] == '+' &&
            stack[top] == 'E') {
            top -= 2;
            stack[top] = 'E';
            stack[top + 1] = '\0';

            printf("%s\t%s\tREDUCE E->E+E\n",
                   stack, input + i);
        }
    }

    if (strcmp(stack, "E") == 0)
        printf("\nString Accepted!\n");
    else
        printf("\nString Rejected!\n");

    return 0;
}
```

```
Writing shift_reduce.c
```

```
%%shell
gcc shift_reduce.c
./a.out
```

```
Enter input string:  id+id

STACK INPUT ACTION
i     d+id  SHIFT
id    +id   SHIFT
E     +id   REDUCE E->id
E+    id    SHIFT
E+i   d     SHIFT
E+id        SHIFT
E+E         REDUCE E->id
E           REDUCE E->E+E

String Accepted!
```

```
%%writefile lr_parser.c
#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c)
{
    stack[++top] = c;
```

```
    stack[top + 1] = '\0';
}

void pop(int n)
{
    top -= n;
    stack[top + 1] = '\0';
}

int main()
{
    char input[100];
    int i = 0;

    printf("Enter input string: ");
    scanf("%s", input);

    printf("\nSTACK\tINPUT\tACTION\n");

    while (input[i] != '\0') {

        /* Shift */
        if (input[i] == 'i' && input[i + 1] == 'd') {
            push('i');
            push('d');
            i += 2;

            printf("%s\t%s\tSHIFT id\n",
                   stack, input + i);

            /* Reduce F -> id */
            pop(2);
            push('F');

            printf("%s\t%s\tREDUCE F->id\n",
                   stack, input + i);

            /* Reduce T -> F */
            pop(1);
            push('T');

            printf("%s\t%s\tREDUCE T->F\n",
                   stack, input + i);

            /* Reduce E -> T */
            if (input[i] == '\0' || input[i] == '+') {
                pop(1);
                push('E');

                printf("%s\t%s\tREDUCE E->T\n",
```

```
                        stack, input + i);
                }
            }

            else if (input[i] == '+') {
                push('+');
                i++;

                printf("%s\t%s\tSHIFT +\n",
                       stack, input + i);
            }

            else {
                printf("\nString Rejected!\n");
                return 0;
            }
        }

        if (strcmp(stack, "E") == 0)
            printf("\nString Accepted!\n");
        else
            printf("\nString Rejected!\n");

        return 0;
}
```

```
Writing lr_parser.c
```

```
%%shell
gcc lr_parser.c
./a.out
```

```
Enter input string: id+id*id

STACK INPUT ACTION
id    +id*id     SHIFT id
F     +id*id     REDUCE F->id
T     +id*id     REDUCE T->F
E     +id*id     REDUCE E->T
E+    id*id SHIFT +
E+id  *id   SHIFT id
E+F   *id   REDUCE F->id
E+T   *id   REDUCE T->F

String Rejected!
```